



KDHS 2022 — Contraceptive Use Prediction

Data Cleaning · EDA · Modelling · SHAP Explainability · Deployment

The Insight Architects Group

Project at a Glance

Dataset	Kenya Demographic and Health Survey (KDHS) 2022 — Women's Individual Recode
Respondents	32,156 women aged 15–49, all 47 Kenyan counties
Target variable	contraceptive_use — four classes (No method · Folkloric · Traditional · Modern)

Methodology	CRISP-DM
Notebook	136 cells
Models trained	5 (Logistic Regression, Random Forest, LightGBM, Deep MLP, TabTransformer)
Selected model	LightGBM — Accuracy 72.4% · Macro-AUC 0.798
Live API	https://kdhs-contraceptive-api.onrender.com
Tools	Python · scikit-learn · LightGBM · PyTorch · SHAP · Optuna · Flask · Render.com

Table of Contents

1. [Why This Project Exists](#)
2. [Dataset Description](#)
3. [Repository Structure](#)
4. [Environment Setup](#)
5. [How to Run the Notebook](#)
6. [Notebook Walkthrough — Section by Section](#)
7. [EDA — Charts and Findings](#)
8. [Feature Engineering Catalogue](#)
9. [The Modelling Process — Step by Step](#)
10. [Model Performance Summary](#)
11. [SHAP Explainability Guide](#)
12. [Key Findings](#)
13. [Business Recommendations](#)
14. [Deployment — Live API](#)
15. [Design Decisions and Honest Limitations](#)

1. Why This Project Exists

Kenya's national family planning statistics tell an encouraging story. Modern contraceptive use among married women has improved over three decades. But national averages hide a reality aggregate reporting cannot surface: in Mandera, barely 1 in 100 women uses a modern method. In Embu, more than half do. That 56-percentage-point gap between two counties in the same country is not a rounding error — it represents a structural inequality in access, information, and agency.

Family planning programmes currently tend to distribute resources uniformly rather than where they are needed most. Without a data-driven way to identify *which specific combination* of sociodemographic characteristics predicts genuine contraceptive non-use, outreach budgets get spread thin and the women who most need contact are hardest to reach.

This project builds a machine learning pipeline that changes that. Using the 2022 Kenya Demographic and Health Survey — the most detailed snapshot of Kenyan women's reproductive health in years — a model is trained to predict which contraceptive method type a woman is currently using, based on who she is and where she lives. The predictions are then explained with SHAP, translated into recommendations policy teams

can act on, and deployed as a live REST API that county health officers can query without writing a line of code.

2. Dataset Description

Source

Field	Detail
Name	Kenya Demographic and Health Survey 2022
Module	Women's Individual Recode (IR file — DHS-7 standard)
Raw file	32,156 rows × 5,925 columns
Working file	32,156 rows × 20 selected columns

Selected Variables

DHS code	Working name	Type	Description
v012	age	Integer	Age in completed years
v013	age_group	Categorical	5-year band: 15–19 through 45–49
v024	county	Categorical	47 Kenyan counties
v025	residence_type	Binary	Urban / Rural
v106	education_level	Ordinal	No education · Primary · Secondary · Higher
v130	religion	Categorical	10 religious affiliations
v136	household_size	Integer	Number of household members
v151	household_head_sex	Binary	Male / Female
v190	wealth_index	Ordinal	DHS wealth quintile: Poorest → Richest
v201	children_ever_born	Integer	Total children ever born
v212	age_first_birth	Float	Age at first birth (structurally missing if no births)
v218	living_children	Integer	Number of children currently alive
v228	pregnancy_loss	Binary	Has ever had a pregnancy loss
v302a	ever_used_contraceptive	Categorical	Contraceptive history — EDA only, dropped before modelling
v313	contraceptive_use	Target	Current method type — 4 classes
v501	marital_status	Categorical	6 categories

DHS code	Working name	Type	Description
v502	union_status	Categorical	Currently / Formerly / Never in union
v701	partner_education	Ordinal	Partner's education level
v714	currently_working	Binary	Currently employed

Target Variable

Class	Share	Notes
No method	58.1%	Dominant majority class
Modern	37.9%	Pills, injection, implant, IUD, condom, sterilisation
Traditional	3.8%	Rhythm, withdrawal, abstinence
Folkloric	0.2% (53 women)	Herbs, amulets, folk remedies

The 53-row Folkloric class is the single most important structural constraint in the entire dataset — it is why SMOTE and macro-averaged metrics are used throughout, and why the model's limitations are stated plainly to stakeholders rather than glossed over.

Missing Values

Column	Missing	Why	Resolution
age_first_birth	8,813	Only asked of women who have given birth	Filled with 0; has_given_birth binary flag created
partner_education	Women with no current partner	Only asked of women with a current partner	Filled with 'No partner' as a valid sixth category

No rows were dropped. All 32,156 respondents are retained throughout — the 274 rows pandas flags as "duplicates" (532 total) all belong to distinct `caseids`, i.e. different women who happen to share the same age/county/education profile.

3. Repository Structure

```

kdhs_project/
├── KDHS_2022_Capstone.ipynb      ← Main notebook (136 cells)
├── KDHS_2022_women.csv          ← Raw dataset – must sit here at runtime
├── README.md                    ← This file
├── deployment artifacts (generated by Section 16 of the notebook)
│   ├── contraceptive_model_bundle.joblib ← Trained LightGBM + preprocessor +
│   │   label encoder (~5.6 MB)
│   └── feature_engineering.py      ← Shared engineer_features() function

```

```
(train/serve parity)
└─ app.py                ← Flask REST API (/health, /predict,
/predict_batch)
```

The three deployment files are written directly from the notebook via `%%writefile` cells in Section 16, and `contraceptive_model_bundle.joblib` is produced at the end of Section 16 once the notebook has run end-to-end.

4. Environment Setup

What you need

Requirement	Notes
Python	3.10+
Jupyter Notebook / JupyterLab	Any current version
RAM	4 GB minimum · 8 GB recommended (SMOTE-expanded training data + PyTorch training)

Install for the full notebook

```
python -m venv venv
source venv/Scripts/activate      # Windows Git Bash
# source venv/bin/activate        # macOS / Linux

pip install pandas numpy matplotlib seaborn \
    scikit-learn lightgbm imbalanced-learn shap \
    torch optuna flask joblib jupyter
```

Notes and gotchas

Issue	Fix
<code>lightgbm</code> fails to install on Windows	Install the Microsoft Visual C++ Redistributable first
<code>torch</code> install is slow / large	Only needed for the MLP and TabTransformer sections — skip those cells if you only want the classical models
<code>imbalanced-learn</code> not found	It's a separate package from scikit-learn and must be installed explicitly
<code>KDHS_2022_women.csv</code> not found at runtime	The data-loading cell uses a relative path — the CSV must sit in the same folder as the notebook

5. How to Run the Notebook

```
jupyter notebook "KDHS_2022_Capstone.ipynb"
```

In Jupyter: **Kernel** → **Restart & Run All**

Important rules:

- `KDHS_2022_women.csv` must be in the same directory as the notebook
- Never run cells out of order — every section builds on variables created by the ones before it (feature engineering depends on cleaning; the split depends on feature engineering; every model depends on the split/SMOTE step; SHAP depends on the trained LightGBM model; deployment depends on all of the above)
- The slowest stages are SHAP computation on 1,500 test respondents, the Optuna hyperparameter search, and PyTorch training for the MLP and TabTransformer — expect these three to dominate total runtime

6. Notebook Walkthrough — Section by Section

The notebook is structured around 16 numbered sections, aligned to CRISP-DM phases.

#	Section	CRISP-DM phase
1	Business Understanding	—
2	Setup & Imports	—
3	Data Understanding	Data Understanding
4	Data Cleaning	Data Preparation
5	Exploratory Data Analysis	Data Understanding
6	Key EDA Findings & Modelling Implications	Data Understanding
7	Feature Engineering	Data Preparation
8	Train / Test Split & Class Imbalance Handling	Data Preparation
9	Model 1 — Logistic Regression	Modelling
10	Model 2 — Random Forest	Modelling
11	Model 3 — LightGBM	Modelling
11b	Model 4 — Deep MLP	Modelling
11c	Model 5 — TabTransformer	Modelling
12	Model Comparison & Selection	Evaluation
13	Hyperparameter Tuning (Optuna)	Evaluation

#	Section	CRISP-DM phase
14	SHAP Explainability	Evaluation
15	Business Recommendations	Evaluation
16	Deployment — Flask API	Deployment

Section 1 — Business Understanding

Sets the scene: Kenya's family planning inequality in plain language, the prediction problem framed against real public health stakes, six stakeholder groups, the pre-registered success metrics, and the five business questions the notebook is designed to answer.

Section 2 — Setup & Imports

Installs SHAP if not already present, then loads every library used across the notebook in one place (pandas, numpy, matplotlib, seaborn, scikit-learn, LightGBM, imbalanced-learn, SHAP, PyTorch) so nothing is imported piecemeal mid-notebook.

Section 3 — Data Understanding

Loads the raw CSV using only the 20 theory-driven columns, checks shape/dtypes/numeric ranges before decoding, previews the raw table, scans for missing values, looks at the target's raw distribution, and investigates the 274 rows flagged as apparent duplicates — confirming via `caseid` that they are genuinely different women, not data errors.

Section 4 — Data Cleaning

Three transparent decisions: **(4.1)** decode every DHS integer code into a plain-English label using the DHS-7 Recode Manual, including mapping all 47 county codes to county names; **(4.2)** handle the two structurally-missing columns (`age_first_birth`, `partner_education`) by flagging rather than imputing invented numbers; **(4.3)** keep all 32,156 rows, since the apparent duplicates are confirmed distinct women. Two assertion checks and a before/after summary table close the section.

Section 5 — Exploratory Data Analysis

The largest section, and the one that shapes everything after it. Three layers: **univariate** (target, age, education, wealth, residence — each alone), **bivariate** (modern-vs-not-modern use by education, wealth, residence, and county), and **multivariate** (education × wealth heatmap; numeric correlation matrix). Every chart is followed by an Insight cell explaining what it means for the modelling decisions ahead. See [Section 7](#) of this README for the full chart-by-chart table.

Section 6 — Key EDA Findings & Modelling Implications

A structured bridge from exploration to modelling: eight findings, each paired with the specific action taken because of it (SMOTE, ordinal encoding, the `arid_county/region` features, and so on).

Section 7 — Feature Engineering

Transforms the cleaned 20-column frame into a 37-column model-ready dataset: drops `caseid` (identifier) and `ever_used_contraceptive` (target leakage), and creates 18 new features, every one traceable to a specific EDA finding. See [Section 8](#) of this README for the full catalogue.

Section 8 — Train / Test Split & Class Imbalance Handling

Stratified 80/20 split, `ColumnTransformer` preprocessing (StandardScaler + OneHotEncoder) fitted on training data only, expanding to 126 model-ready columns, then SMOTE applied to the training set alone, bringing every class up to 14,955 rows. See [Section 9](#) for the full step-by-step breakdown.

Section 9 — Model 1: Logistic Regression

The interpretable linear baseline. `max_iter=2000`, default L2 regularisation. Every subsequent model must beat its AUC (0.764) to justify its added complexity. **Result:** Accuracy 0.548 · Macro-F1 0.354 · Macro-AUC 0.764.

Section 10 — Model 2: Random Forest

300-tree bagging ensemble, `max_depth=14`, `min_samples_leaf=5`, `class_weight='balanced'`. Captures the non-linear wealth cliff and county-level threshold effects the linear model cannot. **Result:** Accuracy 0.706 · **Macro-F1 0.390 (best of all five)** · Macro-AUC 0.775.

Section 11 — Model 3: LightGBM (selected for deployment)

Gradient-boosted trees using histogram-based splitting and leaf-wise growth. `n_estimators=400` · `learning_rate=0.05` · `max_depth=7` · `num_leaves=31` · 80% row/column subsampling. Validation loss plateaus at iteration 226 with no overfitting. **Result: Accuracy 0.724 (best)** · Macro-F1 0.367 · **Macro-AUC 0.798 (best)**.

Section 11b — Model 4: Deep Learning MLP

Four-layer PyTorch network: Linear(126→256)+BatchNorm+ReLU+Dropout → Linear(256→128)+BatchNorm+ReLU+Dropout → Linear(128→64)+BatchNorm+ReLU+Dropout → Linear(64→4). Adam optimiser, 40 epochs, `ReduceLROnPlateau`. **Result:** Accuracy 0.703 · Macro-F1 0.385 · Macro-AUC 0.751.

Section 11c — Model 5: TabTransformer

Self-attention across 13 categorical feature embeddings (16 dimensions each, 2-head attention, 43,044 total parameters). **Result:** Accuracy 0.648 · Macro-F1 0.384 · Macro-AUC 0.754 · **Traditional-class recall 34% (best of all five)**.

Section 12 — Model Comparison & Selection

A side-by-side 5×4 evaluation grid (every model's dashboard in one figure). **LightGBM is selected** — best Macro-AUC, best accuracy, native SHAP support, a 5.6 MB deployable bundle, sub-millisecond CPU inference, no GPU required in production.

Section 13 — Hyperparameter Tuning

Optuna's Tree-structured Parzen Estimator refines LightGBM's parameters, searching on a fast, balanced 15,000-row stratified subsample with 2-fold cross-validation, then retraining the winning configuration on the full dataset. See [Section 9.6](#) for the full step-by-step walkthrough.

Section 14 — SHAP Explainability

`shap.TreeExplainer` on the selected LightGBM model, applied to 1,500 held-out test respondents. Five complementary views — see [Section 11](#) of this README for the full visual guide.

Section 15 — Business Recommendations

Six SHAP-grounded recommendations for the Ministry of Health, County Health Management Teams, UNFPA, and USAID Kenya. Every recommendation cites the specific SHAP weight or EDA finding behind it. See [Section 13](#).

Section 16 — Deployment

Serialises the LightGBM pipeline to `contraceptive_model_bundle.joblib`, writes `feature_engineering.py` and `app.py` via `%%writefile` cells, smoke-tests the API end-to-end locally, and documents the live Render.com deployment and its endpoints.

7. EDA — Charts and Findings

Chart	Section	Visual type	Key finding
Target distribution	5.1	Donut + horizontal bar	58.1% No method · 37.9% Modern · 3.8% Traditional · 0.2% Folkloric — severe imbalance
Age distribution	5.2	Gradient histogram + KDE	Mean age $\approx$ 29, median 28 · concentrated in the 20s, thinning toward 49
Education distribution	5.3	Bar chart	Primary and Secondary $\sim$ 37% each · Higher $\sim$ 15% · No education $\sim$ 12%
Wealth distribution	5.4	Bar chart	Near-uniform across quintiles — roughly 18–22% each
Residence type	5.5	Bar chart	62% rural · 38% urban
Modern use by education	5.6	Grouped bar	No-education: $\sim$ 12% · Primary: 44% · Secondary: 37% (life-stage dip) · Higher: 46%
Modern use by wealth	5.7	Grouped bar	Poorest: $\sim$ 25% → Poorer: $\sim$ 41% (big jump) · flat thereafter — cliff, not gradient
Modern use by residence	5.8	Grouped bar	Urban $\sim$ 38% vs. Rural $\sim$ 38% — essentially identical
Modern use by county	5.9	Diverging lollipop	Embu $\sim$ 57% down to Mandera $\sim$ 1.2% — a 56-point spread across 47 counties

Chart	Section	Visual type	Key finding
Education × Wealth	5.10	Heatmap	Education matters more than wealth — no-education women stay low regardless of wealth quintile
Numeric correlations	5.11	Correlation heatmap	<code>children_ever_born</code> and <code>living_children</code> correlate at $r \approx 0.98$ — near-identical

8. Feature Engineering Catalogue

Eighteen features created — each one traceable to a specific EDA finding. If a specific finding could not be pointed to, the feature was not added.

Feature	Definition	EDA motivation
<code>education_level_ord</code>	No education=0 · Primary=1 · Secondary=2 · Higher=3	Ordinal encoding preserves natural order
<code>wealth_index_ord</code>	Poorest=0 · Poorer=1 · Middle=2 · Richer=3 · Richest=4	Captures the wealth cliff (\$5.7) without inflating feature space with dummies
<code>partner_education_ord</code>	No partner=-1 · No education=0 · ... · Higher=3	Makes partner education numeric; -1 separates "no partner" from "uneducated partner"
<code>age_group_ord</code>	15–19=0 · ... · 45–49=6	Ordered alternative to the 7-level nominal age group
<code>education_gap</code>	<code>partner_edu_ord</code> - <code>edu_ord</code> (0 if no partner)	Within-couple education asymmetry
<code>has_partner</code>	1 if partner education ≠ 'No partner'	Makes the structural missingness in <code>partner_education</code> explicitly binary
<code>child_density</code>	<code>children_ever_born</code> / <code>household_size</code>	Normalises fertility by household context
<code>surviving_ratio</code>	<code>living_children</code> / <code>children_ever_born</code> (1.0 if no births)	Extracts child-mortality signal without re-introducing the $r=0.98$ collinearity
<code>child_loss</code>	<code>children_ever_born</code> - <code>living_children</code>	Direct experience of child loss
<code>is_in_union</code>	1 if <code>union_status</code> = 'Currently in union'	Isolates the union/non-union boundary — the clearest enabler of modern use in SHAP
<code>is_married_or_union</code>	1 if married or living together	Pregnancy-exposure binary for targeting

Feature	Definition	EDA motivation
urban	1 if Urban	Binary recode, friendlier to MLP/Transformer numeric pathways
employed	1 if currently working	Ranks 3rd in global SHAP importance — most actionable top driver
female_hh_head	1 if household head is female	Household gender dynamics
had_pregnancy_loss	1 if ever had a pregnancy loss	Fertility history indicator
age_at_first_birth_gap	age - age_first_birth (0 if no births)	Years since first birth — exposure duration not captured by age_first_birth alone
arid_county	1 if county ∈ {Turkana, West Pokot, Mandera, Wajir, Garissa, Marsabit, Samburu, Isiolo, Tana River}	Operationalises the 56-point county gap (\$5.9)
region	County → 7 Kenyan regions (Coast, North Eastern, Eastern, Central, Rift Valley, Western, Nyanza, Nairobi)	Lower-cardinality geographic alternative to 47-level county

9. The Modelling Process — Step by Step

Step 1 — Define target and feature matrix

`target = 'contraceptive_use'`. The 37-column engineered frame is split into `X` (36 predictors: 22 numeric/ordinal + 14 nominal) and `y`, with `y` label-encoded to integers 0–3 so every model, including LightGBM and the PyTorch models, can consume it identically.

Step 2 — Stratified train/test split

An 80/20 split is performed **before any preprocessing is fitted**, to prevent test-set statistics leaking into the scaler/encoder. Stratification on the target ensures Folkloric (0.2%) and Traditional (3.8%) appear proportionally in both sets — without it, a random split could by chance put all 53 Folkloric respondents in training, leaving none to evaluate on.

Step 3 — Fit preprocessing on training data only

A `ColumnTransformer` combines `StandardScaler` (numeric/ordinal columns) and `OneHotEncoder(handle_unknown='ignore')` (nominal columns), fit only on `X_train` and then applied unchanged to `X_test`. One-hot expansion turns 36 engineered columns into **126 model-ready columns**, driven mostly by the 47-level `county` variable.

Step 4 — Resample the training set with SMOTE (training only)

`SMOTE(random_state=42, k_neighbors=5)` is fit and applied only to `X_train_proc/y_train`. It works by picking a minority-class row, finding its 5 nearest minority-class neighbours, and interpolating new synthetic rows between them. Every class is brought to the majority count of **14,955** rows — Folkloric's 42 original training rows become 14,913 synthetic interpolations. The test set is never touched, which is why macro-F1/macro-AUC on the untouched test set remain the metrics of record.

Step 5 — Train and evaluate five candidate models

Each model trains on the identical SMOTE-resampled training set and is scored on the identical untouched test set:

Model	Mechanics	Configuration	Result
Logistic Regression	Weighted sum of features, one set of weights per class (one-vs-rest)	<code>max_iter=2000</code> , default L2	Acc 0.548 · Macro-F1 0.354 · Macro-AUC 0.764
Random Forest	300 independent trees (bagging), majority vote	<code>max_depth=14</code> · <code>min_samples_leaf=5</code> · <code>class_weight='balanced'</code>	Acc 0.706 · Macro-F1 0.390 · Macro-AUC 0.775
LightGBM	Sequential gradient-boosted trees; histogram-based splitting + leaf-wise growth	<code>n_estimators=400</code> · <code>lr=0.05</code> · <code>max_depth=7</code> · <code>num_leaves=31</code> · 80% subsampling	Acc 0.724 · Macro-F1 0.367 · Macro-AUC 0.798
Deep MLP	4-layer feedforward network with BatchNorm/ReLU/Dropout	Adam, lr 1e-3, 40 epochs, <code>ReduceLROnPlateau</code>	Acc 0.703 · Macro-F1 0.385 · Macro-AUC 0.751
TabTransformer	16-dim categorical embeddings + 2-head self-attention + MLP head	43,044 parameters	Acc 0.648 · Macro-F1 0.384 · Macro-AUC 0.754

Step 6 — Hyperparameter tuning (Optuna)

1. **Subsample for speed:** search runs on a balanced 15,000-row stratified subset rather than the full 59,820-row SMOTE training set (~4× faster per trial).
2. **2-fold CV during search:** enough to rank candidate configurations; full 5-fold precision is reserved for final reporting.

3. **TPE-guided trials:** Optuna's Tree-structured Parzen Estimator proposes a parameter set, evaluates it, and updates its internal model of "good" regions before proposing the next — a Bayesian search rather than blind grid/random search.
4. **Retrain on the full dataset:** the winning parameter set is used to retrain LightGBM on all 59,820 SMOTE-resampled rows.
5. **Validate the gain:** tuned test-set metrics are compared against the manually-configured Section 11 LightGBM to confirm a genuine improvement.

Step 7 — Explain the selected model with SHAP

`shap.TreeExplainer` computes exact Shapley values for the tuned LightGBM model on 1,500 held-out test respondents — see [Section 11](#).

Step 8 — Package and deploy

The trained model, fitted preprocessor, and label encoder are bundled together and served behind a Flask API — see [Section 14](#).

10. Model Performance Summary

All metrics on the **held-out test set**, retaining the natural class imbalance (SMOTE never touches it).

Model	Accuracy	Macro F1	Macro AUC	Modern recall	Traditional recall
LightGBM ★ deployed	0.724	0.367	0.798	71%	~0%
Random Forest	0.706	0.390	0.775	81%	9%
Deep MLP	0.703	0.385	0.751	75%	9%
TabTransformer	0.648	0.384	0.754	70%	34%
Logistic Regression	0.548	0.354	0.764	49%	—

Why LightGBM was chosen: best Macro-AUC (the headline metric, hardest to inflate through resampling) · best accuracy · native SHAP TreeExplainer support · 5.6 MB serialisable bundle · sub-millisecond CPU inference per row · no GPU required in production.

Against the pre-registered success metrics (Section 1):

Target	Achieved	Met?
Accuracy $\geq 78\%$	72.4%	Not met
ROC-AUC ≥ 0.82	0.798	Not met, but close
Macro-F1 ≥ 0.75	0.367	Not met

Honest caveat: no model reliably predicts the Folkloric class (11 test cases). 53 original training examples — even after SMOTE — cannot support generalisation. This is a data constraint, not a tuning problem, and every architecture tried (linear, ensemble, two flavours of deep learning) converges on the same ceiling for Folkloric and Traditional.

11. SHAP Explainability Guide

Computed with `shap.TreeExplainer` on the selected LightGBM model, applied to 1,500 held-out test respondents.

High vs. low feature impact. For each feature, shows how differently the model treats a woman with a high value vs. a low value of that feature — red = high, blue = low, with the gap between them showing how much that characteristic changes the prediction.

Global feature ranking. Mean absolute SHAP value per feature across all four classes and all 1,500 respondents — the definitive importance ranking. Age ranks first, living children second, employment third (the most policy-tractable top driver), followed by arid-county location (a barrier) and household size (a barrier, suggesting resource dilution).

Two representative respondents (waterfall/force plots). Respondent C — predicted "No method" with 99.9% confidence: young, never-married, no children, unemployed — read by the model as *not yet exposed to pregnancy risk*, not as facing an access barrier. Respondent D — predicted "Modern" with 92.5% confidence: older, married, employed, several children, non-arid county — every policy-tractable driver points the same direction.

Per-class breakdown. `age` and `living_children` are universal discriminators (tall bars for every class). `religion_Muslim` and `arid_county` spike specifically for No-method and Folkloric. `employed` is disproportionately large specifically for the Modern class.

Direction-of-influence heatmap. The clearest single policy-reading surface: `is_in_union` is deep red (enabler) for Modern and deep blue (barrier) for No-method — the strongest, most CHP-tractable enabler of modern use. `arid_county` is a structural barrier even after controlling for wealth, education, and religion. `employed` is the most actionable lever in the top tier.

12. Key Findings

From the EDA

Finding	What it means
Modern use ranges from ~57% (Embu) to ~1.2% (Mandera)	Geography is the single strongest signal — the strongest argument for county-level targeting
Wealth shows a cliff at the Poorest/Poorer boundary	The policy lever is pulling women out of the Poorest quintile, not spreading subsidies evenly
Urban and rural women have nearly identical modern-use rates	Residence type alone is a misleading indicator; county, education, and wealth are what actually matter
Secondary-educated women's apparently low modern use is a life-stage effect	They are younger and mostly unmarried, not less receptive to modern methods
<code>children_ever_born</code> and <code>living_children</code> correlate at $r \approx 0.98$	Keeping both creates multicollinearity — <code>surviving_ratio</code> and <code>child_loss</code> are derived instead

From SHAP

Feature	Direction	What it tells us
age	Toward modern use	Older women are more likely to use modern methods — a life-stage effect
living_children	Toward modern use	Women adopt contraception after having the children they want
employed	Toward modern use	Most actionable top driver — employment unlocks access and information
arid_county	Away from modern use	ASAL location is a structural barrier even after controlling for wealth, education, and religion
is_in_union	Toward modern use	Being in a union is the single clearest enabler of modern use

13. Business Recommendations

13.1 Geographic Targeting — Highest Priority

Mandera, Wajir, Garissa, Marsabit, and Tana River should be treated as a distinct, top-priority programme tier. The **arid_county** SHAP signal confirms the gap persists even after controlling for wealth, education, and religion — genuine infrastructure barriers are at work: clinic density, commodity supply chains, and female CHP coverage in remote pastoral areas.

13.2 Target the Right Non-Users

The highest-risk profile for genuine unmet need is: *currently in union + not employed + Poorest wealth quintile + arid county*. Young, never-married, childless women make up a large share of "No method" but largely represent non-exposure to pregnancy risk, not an access problem — conflating the two wastes scarce outreach resources.

13.3 Employment-Linked Programming

employed is the highest-ranked driver that is directly policy-tractable. Co-locating family-planning information with women's economic-empowerment programmes — microfinance groups, vocational training, cooperative societies — reaches the right women where they already gather.

13.4 The Wealth Cliff, Not Gradient

The real wealth effect concentrates in moving women out of the Poorest quintile. Above that threshold the gradient flattens out. Subsidised commodities and outreach should be weighted toward the Poorest quintile specifically.

13.5 Honest Scope Limits

The model reliably discriminates Modern vs. No-method risk. It cannot reliably flag Folkloric or Traditional users at useful precision levels. Stakeholders should frame the output as a binary **Modern / Non-use risk** flag

— not a four-class method-type predictor.

13.6 Deployment Use Case

The API should feed a CHP household-visit prioritisation workflow. A composite flag — *in_union* + *not_employed* + *arid_county* + *Poorest* — derivable from a short community-screening form, gives CHPs a prioritised list without needing an API call at all.

14. Deployment — Live API

Live URL

```
https://kdhs-contraceptive-api.onrender.com
```

Endpoints

Route	Method	Auth	Description
/	GET	None	HTML form for non-technical users
/health	GET	None	Liveness check
/predict	POST	X-API-Key header	Single respondent → prediction + probabilities
/predict_batch	POST	X-API-Key header	JSON array → list of predictions

Required JSON payload

All 17 fields required, using decoded string labels rather than DHS integer codes.

```
{
  "age": 28,
  "age_group": "25-29",
  "county": "Turkana",
  "residence_type": "Rural",
  "education_level": "Primary",
  "religion": "Roman Catholic",
  "household_size": 6,
  "household_head_sex": "Male",
  "wealth_index": "Poorest",
  "children_ever_born": 3,
  "age_first_birth": 19,
  "living_children": 3,
  "pregnancy_loss": "No",
  "marital_status": "Married",
  "union_status": "Currently in union",
  "partner_education": "Primary",
```

```
"currently_working": "No"
}
```

Example response (smoke-tested in the notebook)

```
{
  "prediction": "No method",
  "probabilities": {
    "Folkloric": 0.0003,
    "Modern": 0.406,
    "No method": 0.522,
    "Traditional": 0.0717
  },
  "risk_flag": "non_use_risk"
}
```

`risk_flag` is `"non_use_risk"` when the prediction is "No method"; `"using_method"` for any other class. This exact test case — a married, in-union, unemployed woman in Poorest-quintile Turkana — matches what the EDA and SHAP analysis identify as the primary drivers of non-use.

Run locally

```
export API_KEY="your-secret-key"
python app.py
```

Open <http://localhost:5000/> in a browser, or test via curl:

```
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -H "X-API-Key: your-secret-key" \
  -d @sample_request.json
```

Redeployment

Render.com builds a Docker container directly from the connected GitHub repository, restarts automatically on crash, and redeploys automatically on every push:

```
git add .
git commit -m "describe your change"
git push
# Render detects the push and redeploys in ~2 minutes
```

Free-tier behaviour

Render's free tier spins the service down after 15 minutes of inactivity. The first request after a sleep takes 30–60 seconds to wake; subsequent requests are instant. The Starter plan (\$7/month) removes this limitation.

Why `feature_engineering.py` is a separate shared file

The same `engineer_features(record)` function is used at training time (in the notebook) and at inference time (in `app.py`). This makes it structurally impossible for the notebook and the deployed API to compute features differently — the most common cause of silent model degradation in production.

15. Design Decisions and Honest Limitations

Why we made the choices we did

Decision	Reasoning
LightGBM over the other four models	Best Macro-AUC, best accuracy, native SHAP support, tiny bundle, no GPU dependency in production
SMOTE on training set only	The test set must reflect the real-world imbalance so reported metrics mean something
Macro-AUC as headline metric	Accuracy is easily inflated by predicting the majority class; Macro-AUC evaluates probability calibration across all four classes equally
<code>ever_used_contraceptive</code> dropped before modelling	It sits in the same contraceptive-history survey module as the target — keeping it would create near-tautological leakage
<code>feature_engineering.py</code> as a shared module	Prevents train/serve feature skew — the most common cause of production ML failures
Modern-vs-not-modern reframing for the bivariate EDA	Tracks the mCPR metric MoH/UNFPA/FP2030 actually monitor, and avoids drawing conclusions from Folkloric's 53 cases

What we are honest about

- The Folkloric class cannot be predicted reliably.** 53 original training examples — even after SMOTE generates thousands of synthetic variants — cannot provide enough genuine variation for any model to generalise. Every model scores near-zero recall on the 11 Folkloric test cases. This is not fixable with better hyperparameters; it requires more data.
- The pre-registered success metrics were not fully met.** Accuracy $\geq 78\%$ (achieved 72.4%), Macro-AUC ≥ 0.82 (achieved 0.798), Macro-F1 ≥ 0.75 (achieved 0.367). All shortfalls trace to the Folkloric/Traditional long tail. On the binary Modern-vs-No-method problem the model performs well: Modern AUC = 0.823, No-method AUC = 0.834.
- High non-use risk does not always mean unmet need.** Young, never-married, childless women have low modern-use rates largely because they have low pregnancy exposure, not because they face access barriers. The risk flag should always be interpreted alongside union status before any outreach decision.

Correlation is not causation. The KDHS is a cross-sectional survey, not a randomised trial. The association between employment and modern-method use may reflect confounding factors. All recommendations should be treated as evidence-informed hypotheses, not proven interventions.

The Insight Architects KDHS 2022 Capstone Live API: <https://kdhs-contraceptive-api.onrender.com>